A 3.3.2 Constructing Queries in SQL

A 3.3.2 Construct queries between two tables in SQL.

Querying across two SQL tables involves using JOIN to combine data. Essential elements include relational and logical operators for filtering, LIKE for pattern matching, and ORDER BY for sorting. Key commands are SELECT, FROM, WHERE, JOIN, and more.

Constructing Queries Between Two Tables in SQL

In relational databases, data is often spread across multiple tables to ensure efficiency and reduce redundancy.

The power of SQL lies in its ability to retrieve and combine this related data through well-constructed queries.

This chapter will guide you through the essential techniques for querying data from two tables, focusing on joins, filtering, pattern matching, and ordering.

Understanding the Need for Multi-Table Queries

Imagine a database for a school. You might have one table containing student information (name, ID, date of birth) and another table listing the courses each student is enrolled in (student ID, course name, grade).

To answer questions like "What are the names of the students enrolled in 'Computer Science'?", you need to access information from both tables and link them based on a common attribute, in this case, the student ID.

Core Concepts: Tables, Keys, and Relationships

Before we dive into constructing queries, let's briefly recap some fundamental database concepts:

- **Tables:** These are the fundamental building blocks of a relational database, organized into rows (records) and columns (attributes).
- **Primary Key:** A column or a set of columns that uniquely identifies each row within a table.

- **Foreign Key:** A column or a set of columns in one table that refers to the primary key in another table. Foreign keys establish and enforce relationships between tables.
- **Relationships:** These define how data in one table is related to data in another table. Common types include one-to-one, one-to-many, and many-to-many.

Understanding these concepts is crucial because you will typically join tables based on the relationship established through primary and foreign keys.

The `JOIN` Clause: Combining Data from Multiple Tables

The `JOIN` clause is the cornerstone of querying data from two or more tables. It allows you to combine rows from different tables based on a related column between them. There are several types of `JOIN` clauses, each serving a specific purpose:

`INNER JOIN`

An `INNER JOIN` (or simply `JOIN`) returns rows only when there is a match in both tables based on the specified join condition.

Syntax:

```
SELECT column1, column2, ...
FROM table1
INNER JOIN table2 ON table1.joining_column = table2.joining_column;
```

- `SELECT column1, column2, ...`: Specifies the columns you want to retrieve from the joined tables. You might need to prefix column names with the table name if the same column name exists in both tables (e.g., `table1.student_id`).
- `FROM table1`: The first table you want to join.
- `INNER JOIN table2`: The second table you want to join with `table1`.
- `ON table1.joining_column = table2.joining_column`: This is the join condition. It specifies the columns that should be compared to find matching rows. Typically, this involves comparing the primary key of one table with the foreign key in the other table.

Example:

Consider our `Students` table with columns `StudentID` (primary key), `Name`, and `DOB`, and a `Enrollments` table with columns `EnrollmentID` (primary key), `StudentID` (foreign key referencing `Students.StudentID`), and `CourseName`.

```
SELECT s.Name, e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID;
```

This query will return the names of students and the courses they are enrolled in. Only students who have a corresponding entry in the `Enrollments` table will be included in the result.

`LEFT JOIN` (or `LEFT OUTER JOIN`)

A `LEFT JOIN` returns all rows from the left table (the table mentioned first in the `FROM` clause) and the matching rows from the right table (the table mentioned after `LEFT JOIN`). If there is no match in the right table, `NULL` values will be returned for the columns of the right table.

Syntax:

```
SELECT column1, column2, ...
FROM left_table
LEFT JOIN right_table ON left_table.joining_column = right_table.joining_column;
```

Example:

```
SELECT s.Name, e.CourseName
FROM Students s
LEFT JOIN Enrollments e ON s.StudentID = e.StudentID;
```

This query will return all students from the `Students` table, along with their enrolled courses if they exist in the `Enrollments` table. If a student is not enrolled in any course, their name will still be displayed, but the `CourseName` will be `NULL`.

`RIGHT JOIN` (or `RIGHT OUTER JOIN`)

A `RIGHT JOIN` is similar to a `LEFT JOIN` but returns all rows from the right table and the matching rows from the left table. If there is no match in the left table, `NULL` values will be returned for the columns of the left table.

Syntax:

```
SELECT column1, column2, ...
FROM left_table
RIGHT JOIN right_table ON left_table.joining_column = right_table.joining_column;
```

Example:

```
SELECT s.Name, e.CourseName
FROM Students s
RIGHT JOIN Enrollments e ON s.StudentID = e.StudentID;
```

This query will return all enrollments from the `Enrollments` table, along with the names of the students if they exist in the `Students` table. If an enrollment has a `StudentID` that doesn't exist in the `Students` table (which ideally shouldn't happen due to referential integrity), the `Name` will be `NULL`.

`FULL OUTER JOIN` (or `FULL JOIN`)

A `FULL OUTER JOIN` returns all rows from both the left and the right tables. It includes matching rows where the join condition is met, and `NULL` values for columns of the table that does not have a match.

Syntax:

```
SELECT column1, column2, ...
FROM left_table
FULL OUTER JOIN right_table ON left_table.joining_column = right_table.joining_co
```

Example:

```
SELECT s.Name, e.CourseName
FROM Students s
FULL OUTER JOIN Enrollments e ON s.StudentID = e.StudentID;
```

This query will return all students and all enrollments. If a student has no enrollments, their name will be shown with `NULL` for `CourseName`. If an enrollment has a `StudentID` not in the `Students` table, the `CourseName` will be shown with `NULL` for `Name`.

Note: The availability of `FULL OUTER JOIN` can vary across different database systems.

Relational Operators: Comparing Values

Relational operators are used in the `WHERE` clause (and sometimes in the `ON` clause of a `JOIN`) to compare values and filter data. Common relational operators include:

- `=` (equal to)
- `>` (greater than)
- `<` (less than)
- `>=` (greater than or equal to)
- `<=` (less than or equal to)
- `!=` or `<>` (not equal to)

Example:

To find the names of students enrolled in courses where the `EnrollmentID` is greater than 5:

```
SELECT s.Name, e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.EnrollmentID > 5;
```

Filtering Data with the `WHERE` Clause

The `WHERE` clause is used to specify conditions that rows must satisfy to be included in the query result. When querying multiple tables, you can use the `WHERE` clause to filter based on columns from any of the joined tables.

Example:

To find the names and courses of students named 'Alice':

```
SELECT s.Name, e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE s.Name = 'Alice';
```

The `BETWEEN` Operator

The `BETWEEN` operator is used to filter rows based on a range of values (inclusive).

Syntax:

```
WHERE column BETWEEN value1 AND value2;
```

Example:

To find enrollments with `EnrollmentID` between 3 and 7:

```
SELECT s.Name, e.CourseName, e.EnrollmentID
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.EnrollmentID BETWEEN 3 AND 7;
```

Pattern Matching with the `LIKE` Clause

The `LIKE` clause is used to search for patterns in string columns. It is often used with wildcard characters:

- `%`: Represents zero or more characters.
- `_`: Represents a single character.

Example:

To find the names of students whose names start with 'A':

```
SELECT Name
FROM Students
WHERE Name LIKE 'A%';
```

To find the names of courses that contain the word 'Science':

```
SELECT s.Name, e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.CourseName LIKE '%Science%';
```

Logical Operators: Combining Conditions (`AND`, `OR`, `NOT`)

Logical operators allow you to combine multiple conditions in the `WHERE` clause:

- `AND`: Both conditions must be true for the row to be included.
- `OR`: At least one of the conditions must be true.
- `NOT`: The condition must be false.

Example:

To find students named 'Alice' who are enrolled in 'Computer Science':

```
SELECT s.Name, e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE s.Name = 'Alice' AND e.CourseName = 'Computer Science';
```

To find students named 'Alice' or students enrolled in 'Physics':

```
SELECT s.Name, e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE s.Name = 'Alice' OR e.CourseName = 'Physics';
```

To find students not enrolled in 'Mathematics':

```
SELECT s.Name, e.CourseName
FROM Students s
LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.CourseName IS NULL OR NOT e.CourseName = 'Mathematics';
```

Ordering Data with the `ORDER BY` Clause

The `ORDER BY` clause is used to sort the results of a query based on one or more columns.

Syntax:

```
SELECT column1, column2, ...
FROM table1
JOIN table2 ON condition
WHERE condition
ORDER BY column_to_sort [ASC | DESC], other_column [ASC | DESC], ...;
```

- `ORDER BY column_to_sort`: Specifies the column(s) to sort by.
- `ASC`: Sorts in ascending order (default).
- `DESC`: Sorts in descending order.

Example:

To retrieve student names and their enrolled courses, ordered alphabetically by student name and then by course name:

```
SELECT s.Name, e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
ORDER BY s.Name ASC, e.CourseName ASC;
```

Selecting Specific Columns with `SELECT` and `DISTINCT`

Remember that the `SELECT` clause determines which columns are included in the result set. When querying multiple tables, be explicit about which table each column belongs to (using aliases like `s.Name` and `e.CourseName` as shown in the examples).

The `DISTINCT` keyword can be used to retrieve only unique rows from the combined result.

Example:

To find the unique course names that students are enrolled in:

```
SELECT DISTINCT e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID;
```

Comprehensive Example

Let's solidify our understanding with a more complex example. Suppose we want to find the names of students who are enrolled in courses containing the word 'Science', and we want to order the results by student name in descending order.

```
SELECT s.Name, e.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.CourseName LIKE '%Science%'
ORDER BY s.Name DESC;
```

This query first joins the `Students` and `Enrollments` tables based on `StudentID`. Then, it filters the results to include only enrollments where the `CourseName` contains 'Science'. Finally, it orders the resulting student names in reverse alphabetical order.

Summary

Constructing queries between two tables in SQL is a fundamental skill for anyone working with relational databases. By mastering the different types of `JOIN` clauses, understanding how to filter data using the `WHERE` clause with relational and logical operators, applying pattern matching with `LIKE`, and ordering your results with `ORDER BY`, you can effectively retrieve and combine information from related tables to answer complex data-driven questions. While the core SQL commands are generally consistent, minor syntax variations might exist between different database systems.

Report a problem